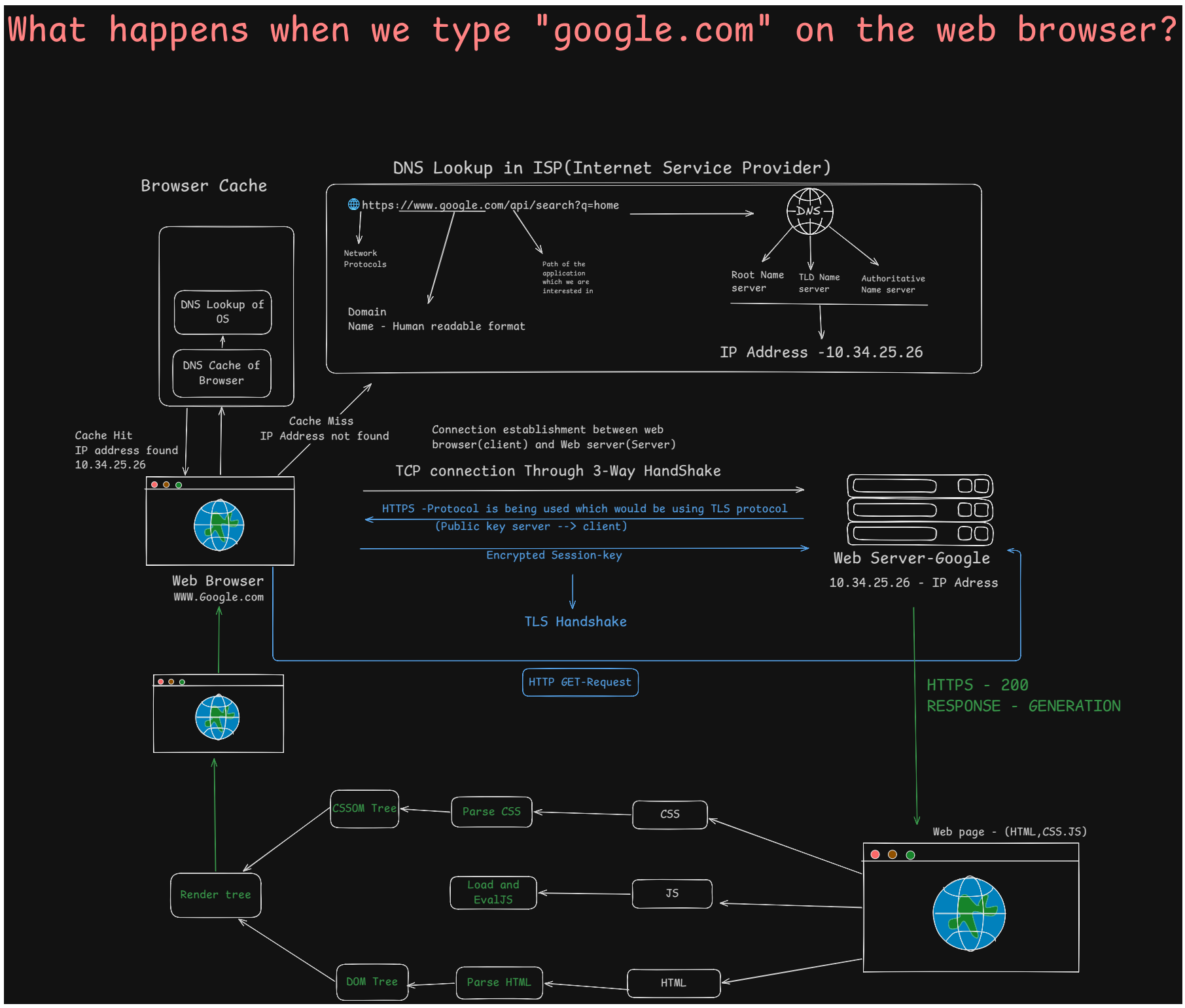
1.How does Internet work



Step 1: You Press Enter – Browser Parses the URL

The browser first breaks down what you typed:

- **Protocol**: `https://`
- **Domain**: `google.com`
- **Path**: `/` (default root)

  It checks if this is a valid URL or a search query. Since `google.com` is a valid domain, it proceeds as a URL.

Step 2: DNS Resolution – "Where is google.com?"

The internet doesn't understand domain names – it only understands **IP addresses** (like `142.250.195.46`). So the browser needs to **translate** `google.com` into an IP address. This is done via **DNS (Domain Name System)**. Here's the DNS lookup chain:

1. Browser Cache

The browser first checks its **own DNS cache**. If you visited google.com recently, it may already know the IP.

2. OS Cache

If not found, it asks the **Operating System**, which also maintains a DNS cache. On Linux/Mac this is checked via `/etc/hosts` file too.

3. Router Cache

If the OS doesn't know, the request goes to your **local router**, which may have the answer cached.

4. ISP's Recursive Resolver

If the router doesn't know, the request goes to your **ISP's DNS Resolver** (a recursive resolver). This resolver does the heavy lifting:

- It contacts the **Root DNS Server** → which says "I don't know google.com, but I know who handles `.com` — go ask the TLD server."
- It contacts the **TLD (Top Level Domain) Server** for `.com` → which says "I don't know google.com specifically, but go ask Google's authoritative name server."
- It contacts **Google's Authoritative DNS Server** → which finally returns the IP address: e.g., `142.250.195.46`

This IP is then **cached** at every level (with a TTL — Time To Live) so future lookups are faster.

Step 3: TCP Connection — "Let's establish a communication channel"

Now the browser knows the IP. But before any data is sent, it needs to establish a **reliable connection** using **TCP (Transmission Control Protocol)**.
This happens via the **TCP 3-Way Handshake**:

```
Client ——— SYN ————————→ Server   (Hey, can we talk?)
Client ←——— SYN-ACK ————— Server   (Sure, I'm ready!)
Client ——— ACK ————————→ Server   (Great, let's go!)
```

- **SYN** → Client sends a synchronize packet
- **SYN-ACK** → Server acknowledges and sends its own sync
- **ACK** → Client acknowledges back

Now a reliable connection exists between your browser and Google's server.

Step 4: TLS Handshake — "Let's make it secure"

Since the URL is `https`, the connection must be **encrypted** using **TLS (Transport Layer Security)**. This happens right after TCP.

The TLS handshake:

1. Browser tells the server which **TLS version and encryption algorithms** it supports.
2. Server responds with its **SSL Certificate** (issued by a trusted Certificate Authority like DigiCert).
3. Browser **verifies** the certificate — checks it's valid, not expired, and trusted.
4. They agree on a **session key** for symmetric encryption.
5. All further communication is **encrypted**.

This is why you see the 🔒 padlock in the browser — your connection is secure.

Step 5: HTTP Request — "Give me the Google homepage"

Now the browser sends an **HTTP GET request** over the secure connection:

```
GET / HTTP/1.1
Host: www.google.com
User-Agent: Chrome/120
Accept: text/html
```

This request travels from your device → through your router → through your ISP → across the internet backbone → to Google's data center.

Step 6: Google's Infrastructure — What happens on Google's side?

This is where it gets interesting from a **system design** perspective:

Load Balancer

Your request doesn't go directly to one server. Google has thousands of servers. A **Load Balancer** receives your request first and routes it to the most appropriate, least busy server.

Web Server / Application Server

The selected server processes your request. For the homepage, it generates the HTML dynamically (personalized based on your location, cookies, login state, etc.).

Caching Layer

Frequently requested resources (scripts, images, styles) are served from a **cache** (like Redis or Memcached) rather than being recomputed every time — making responses much faster.

CDN (Content Delivery Network)

Static assets (JavaScript, CSS, images) are often served from a **CDN node closest to you geographically** — so if you're in Bengaluru, Google serves assets from a nearby edge server rather than from the US.

Step 7: HTTP Response — "Here's the page"

Google's server sends back an **HTTP Response**:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Encoding: gzip
...
<html> ... </html>
```

This response travels back to your browser.

Step 8: Browser Renders the Page

Now the browser does a lot of work to turn raw HTML into what you see:

1. **HTML Parsing** → Builds the **DOM (Document Object Model)** tree
2. **CSS Parsing** → Builds the **CSSOM (CSS Object Model)** tree
3. **Render Tree** → Combines DOM + CSSOM
4. **JavaScript Execution** → JS engine (V8 in Chrome) runs scripts, which may modify the DOM
5. **Layout** → Browser calculates the position/size of every element
6. **Painting** → Pixels are drawn on screen
7. **Additional Requests** → The HTML references other resources (images, fonts, JS files) — the browser fires off more HTTP requests to fetch these

Key Concepts Covered Here

Concept	Role
DNS	Translates domain to IP
TCP	Reliable connection protocol
TLS/SSL	Encryption & security
HTTP/HTTPS	Application-level communication
Load Balancer	Distributes traffic across servers
CDN	Serves static assets from nearby nodes
Caching	Speeds up repeated data access
Browser Rendering	Turns HTML/CSS/JS into a visual page